

RotateToHub: Feedforward & PID Control

Aditya Zaveri

March 7, 2026

Step 1: State Acquisition

At the beginning of the `execute()` loop, the robot must acquire its current physical state on the field. This math is all field-centric.

- **Position:** The robot's current coordinates (x_r, y_r) and heading θ .
- **Velocity:** The robot's current field-centric velocities (v_x, v_y) .
- **Target:** The target's coordinates (x_h, y_h) .

The spatial differences between the target and the robot are defined as:

$$\begin{aligned}\Delta x &= x_h - x_r \\ \Delta y &= y_h - y_r \\ d^2 &= \Delta x^2 + \Delta y^2\end{aligned}$$

Step 2: Kinematic Feedforward (Predictive)

Feedforward answers the question: *"Based purely on physics, how fast do the wheels need to spin right now to keep the Hub centered, assuming zero error?"*

The required angular velocity (ω_{FF}) to track a point while translating is derived from the cross product of the position vector $\vec{r}$ and the velocity vector $\vec{v}$.

Derivation

The angular velocity scalar is defined as the magnitude of the velocity component perpendicular to the position vector, divided by the distance:

$$\omega = \frac{|\vec{v}_\perp|}{|\vec{r}|} = \frac{|\vec{r} \times \vec{v}|}{|\vec{r}|^2} \quad (1)$$

Expanding the 2D cross product $\vec{r} \times \vec{v}$ where $\vec{r} = \langle \Delta x, \Delta y \rangle$ and $\vec{v} = \langle v_x, v_y \rangle$:

$$\omega_{FF} = \frac{\Delta x \cdot v_y - \Delta y \cdot v_x}{\Delta x^2 + \Delta y^2} \quad (2)$$

Sign Convention Correction

In standard FRC coordinates (CCW+), rotating left is positive. If the Hub is straight ahead on the $+X$ axis ($\Delta x > 0, \Delta y = 0$), and the robot translates to the left ($+Y$ axis, so $v_y > 0$), the Hub is effectively moving to the right relative to the robot. Therefore, the robot must rotate right (a **negative** angular velocity) to track it.

Plugging this into our cross product:

$$\Delta x(v_y) - 0(0) = \text{Positive} \quad (\text{Incorrect sign for CCW+})$$

To correct for the coordinate system and ensure the robot turns *toward* the shifting target rather than away from it, the terms are negated:

$$\omega_{FF} = \frac{v_x \Delta y - v_y \Delta x}{d^2} \quad (3)$$

Step 3: Proportional Control (Reactive)

Because physical systems are imperfect (carpet friction, delayed motor response, wheel slip), the Feedforward alone will slowly drift. A standard PID controller is layered on top to correct these physical disturbances.

Because the setpoint (the angle to the Hub) is constantly changing as the robot moves, a `ProfiledPIDController` is deprecated in favor of a standard `PIDController`. A profiled controller recalculates a trapezoidal motion path every 20ms when the target moves, permanently restricting acceleration and causing artificial lag.

The Proportional Output (ω_{PID}) is calculated as:

$$\omega_{PID} = P \times (\theta_{target} - \theta_{current}) \quad (4)$$

Physical Meaning of P : The Proportional gain P represents the conversion factor between angular error and motor effort. For example, a P value of 5.0 means: *"For every 1 radian of rotational error, command an additional 5.0 radians per second of rotational velocity to correct it."*

Step 4: Combination and Hardware Constraints

The predictive mathematical baseline is summed with the reactive error correction to produce the final commanded velocity.

$$\omega_{total} = \omega_{FF} + \omega_{PID} \quad (5)$$

To protect the physical hardware (e.g., MK4i modules) from requesting unachievable speeds, the output is clamped to the physical maximum capabilities of the drivetrain before being dispatched to the Swerve API and PathPlanner overrides.

$$\omega_{capped} = \max(-\omega_{max}, \min(\omega_{total}, \omega_{max})) \quad (6)$$